

Programação Orientada a Objetos

GCT052 - Aula 4.0 - Herança

Prof. Dr. Bento Rafael Siqueira

Universidade Federal de Lavras
Departamento de Ciência da Computação

17 de novembro de 2025

Sumário

- 1 Introdução à Herança
- 2 Conceitos Fundamentais
- 3 Modificadores de Acesso
- 4 Sobrescrita de Métodos
- 5 Exemplos Práticos
- 6 Implementação em C#
- 7 Exemplos de Código
- 8 Exercícios Práticos
- 9 Boas Práticas
- 10 Próximos Passos

O que é Herança?

- **Herança:** Mecanismo que permite criar novas classes baseadas em classes existentes
- **Classe Base (Pai):** Classe que fornece características e comportamentos
- **Classe Derivada (Filha):** Classe que herda e pode estender ou modificar
- **Relacionamento "é um":** Classe filha é um tipo especial da classe pai

Analogia do Mundo Real:

- **Classe Base:** Animal
- **Classes Derivadas:** Cachorro, Gato, Pássaro
- **Relacionamento:** Cachorro é um Animal, Gato é um Animal

Por que usar Herança?

- 1 **Reutilização de Código:** Evita duplicação
- 2 **Organização:** Hierarquia clara e lógica
- 3 **Manutenção:** Mudanças na classe base afetam todas as derivadas
- 4 **Polimorfismo:** Permite tratar objetos de forma uniforme
- 5 **Extensibilidade:** Fácil adicionar novos tipos

Vantagens:

- Código mais limpo e organizado
- Facilita manutenção e evolução
- Permite modelagem hierárquica do mundo real
- Base para polimorfismo

Terminologia da Herança

- **Classe Base (Superclasse):** Classe que é herdada
- **Classe Derivada (Subclasse):** Classe que herda
- **Herança Simples:** Uma classe pode herdar de apenas uma classe base
- **Herança Múltipla:** Não suportada em C# (mas interfaces sim)
- **Modificador protected:** Acessível na classe e nas derivadas

Exemplo:

- **Classe Base:** Veiculo
- **Classe Derivada:** Carro : Veiculo
- **Relacionamento:** Carro é um Veículo

Sintaxe de Herança em C#

Sintaxe básica:

- `public class ClasseDerivada : ClasseBase`
- `public class Cachorro : Animal`
- `public class Carro : Veiculo`

Características:

- Usa dois pontos (:) para indicar herança
- Classe derivada herda todos os membros públicos e protegidos
- Pode adicionar novos membros
- Pode sobrescrever métodos virtuais
- Pode chamar construtor da classe base com `base()`

Modificadores e Herança

- 1 **public**: Acessível de qualquer lugar
- 2 **private**: Acessível apenas na própria classe
- 3 **protected**: Acessível na classe e nas classes derivadas
- 4 **internal**: Acessível no mesmo assembly
- 5 **protected internal**: Acessível no assembly ou classes derivadas

Boas Práticas:

- Use **protected** para membros que classes derivadas precisam acessar
- Mantenha encapsulamento com **private**
- Use **public** para interface pública

Construtores e Herança

- **Construtor da Base:** Pode ser chamado com `base()`
- **Ordem de Execução:** Base primeiro, depois derivada
- **Construtor Padrão:** Se não especificar, chama construtor sem parâmetros da base
- **Construtor Específico:** Deve chamar explicitamente com `base()`

Exemplo:

- `public Cachorro(string nome) : base(nome, "Cachorro")`
- Primeiro executa construtor de `Animal`
- Depois executa construtor de `Cachorro`

Métodos Virtuais e Override

- **virtual**: Permite que método seja sobrescrito
- **override**: Sobrescreve método virtual da classe base
- **sealed**: Impede que método seja sobrescrito novamente
- **new**: Oculta membro da classe base (não recomendado)

Exemplo:

- `public virtual void FazerSom()` na classe base
- `public override void FazerSom()` na classe derivada
- Cada classe pode ter sua própria implementação

Polimorfismo

- **Polimorfismo:** Capacidade de objetos diferentes responderem de forma diferente à mesma mensagem
- **Ligação Tardia:** Decisão sobre qual método chamar é feita em tempo de execução
- **Referência da Base:** Pode apontar para objeto da derivada
- **Comportamento Específico:** Método sobrescrito é chamado

Exemplo:

- `Animal animal = new Cachorro();`
- `animal.FazerSom();` // Chama método de Cachorro
- Mesma referência, comportamentos diferentes

Exemplo 1: Hierarquia de Animais

Estrutura:

- **Classe Base:** Animal (nome, especie, idade, peso)
- **Classe Derivada 1:** Cachorro : Animal
- **Classe Derivada 2:** Gato : Animal
- **Classe Derivada 3:** Passaro : Animal

Características:

- Todos herdam atributos e métodos de Animal
- Cada um sobreescreve FazerSom() de forma diferente
- Cada um pode ter métodos específicos

Exemplo 2: Hierarquia de Veículos

Estrutura:

- **Classe Base:** Veiculo (marca, modelo, ano)
- **Classe Derivada 1:** Carro : Veiculo
- **Classe Derivada 2:** Moto : Veiculo
- **Classe Derivada 3:** Caminhao : Veiculo

Características:

- Todos têm características comuns de veículo
- Cada tipo tem comportamentos específicos
- Método Acelerar() pode ser sobrescrito

Exemplo 3: Hierarquia de Funcionários

Estrutura:

- **Classe Base:** Funcionario (nome, salario, matricula)
- **Classe Derivada 1:** Gerente : Funcionario
- **Classe Derivada 2:** Desenvolvedor : Funcionario
- **Classe Derivada 3:** Analista : Funcionario

Características:

- Todos têm dados básicos de funcionário
- Cada cargo tem responsabilidades específicas
- Método `CalcularSalario()` pode ser sobrescrito

Estrutura Básica de Herança

Elementos principais:

- **Classe Base:** Define membros comuns
- **Classe Derivada:** Herda e estende funcionalidades
- **Construtores:** Chamam construtor da base com `base()`
- **Métodos Virtuais:** Permitem sobrescrita
- **Métodos Override:** Implementam comportamento específico

Exemplo completo: Ver arquivos `Exemplos/`

Padrões de Implementação

Classe Base:

- Atributos protegidos ou privados
- Métodos virtuais para comportamentos variáveis
- Construtores para inicialização
- Métodos comuns a todas as derivadas

Classe Derivada:

- Herda membros da base
- Adiciona atributos específicos
- Sobrescreve métodos virtuais
- Chama construtor da base

Exemplo Completo: Classe Animal (Base)

Arquivo: Exemplos/Animal.cs

- Classe base com atributos comuns
- Métodos virtuais para comportamentos variáveis
- Construtor para inicialização
- Properties para acesso controlado

Características:

- Método virtual `void FazerSom()`
- Método virtual `void Mover()`
- Atributos protegidos para acesso das derivadas
- Código bem documentado

Exemplo Completo: Classe Cachorro (Derivada)

Arquivo: Exemplos/Cachorro.cs

- Herda de Animal
- Sobrescreve FazerSom() com "Au au!"
- Adiciona método específico Latir()
- Usa construtor da base

Características:

- `public class Cachorro : Animal`
- `public override void FazerSom()`
- Comportamento específico de cachorro
- Mantém funcionalidades da base

Exemplo Completo: Classe Gato (Derivada)

Arquivo: Exemplos/Gato.cs

- Herda de Animal
- Sobrescreve FazerSom() com "Miau!"
- Adiciona método específico Ronronar()
- Comportamento único de gato

Características:

- Mesma estrutura de Cachorro
- Comportamento diferente
- Demonstra polimorfismo

Exercício 1: Hierarquia de Formas Geométricas

Crie uma hierarquia de formas:

Classes necessárias:

- Forma (base) - cor, área, perímetro
- Retangulo : Forma - largura, altura
- Circulo : Forma - raio
- Triangulo : Forma - base, altura

Requisitos:

- Método virtual CalcularArea()
- Método virtual CalcularPerimetro()
- Cada forma calcula de forma diferente

Tempo: 30 minutos

Exercício 2: Hierarquia de Contas Bancárias

Crie uma hierarquia de contas:

Classes necessárias:

- ContaBancaria (base) - numero, titular, saldo
- ContaCorrente : ContaBancaria - limite, taxa
- ContaPoupanca : ContaBancaria - rendimento
- ContaInvestimento : ContaBancaria - tipo investimento

Requisitos:

- Método virtual Sacar() com regras diferentes
- Método virtual CalcularRendimento()
- Cada tipo tem regras específicas

Tempo: 35 minutos

Exercício 3: Hierarquia de Produtos

Crie uma hierarquia de produtos:

Classes necessárias:

- Produto (base) - código, nome, preço
- Livro : Produto - autor, isbn, páginas
- Eletrônico : Produto - garantia, voltagem
- Roupas : Produto - tamanho, cor, material

Requisitos:

- Método virtual `CalcularDesconto()`
- Método virtual `ExibirInformacoes()`
- Cada tipo tem informações específicas

Tempo: 40 minutos

Design de Herança

- **Use herança para relacionamento "é um"**
- **Evite herança profunda** (máximo 3-4 níveis)
- **Prefer composição sobre herança** quando apropriado
- **Use classes abstratas** para classes base que não devem ser instanciadas
- **Documente** a hierarquia de herança

Princípios:

- **Princípio da Substituição de Liskov:** Objetos derivados devem poder substituir objetos base
- **Open/Closed Principle:** Aberto para extensão, fechado para modificação
- **DRY:** Don't Repeat Yourself - reutilize código

Implementação em C#

- **Use** `virtual` para métodos que podem ser sobrescritos
- **Use** `override` para sobrescrever métodos virtuais
- **Use** `base()` para chamar construtor da base
- **Use** `protected` para membros acessíveis às derivadas
- **Documente** com XML comments

Evite:

- Herança apenas para reutilizar código
- Classes base muito grandes
- Hierarquias muito profundas
- Uso de `new` para ocultar membros

Testes e Validação

- **Teste** cada classe derivada independentemente
- **Valide** que métodos sobrescritos funcionam corretamente
- **Teste** polimorfismo com referências da base
- **Verifique** que construtores são chamados na ordem correta

Validações importantes:

- Herança mantém comportamento esperado
- Sobrescrita funciona corretamente
- Polimorfismo se comporta como esperado
- Encapsulamento é respeitado

Tópicos Relacionados

1 Classes Abstratas

- Classes que não podem ser instanciadas
- Métodos abstratos obrigatórios
- Base para hierarquias

2 Interfaces

- Contratos de comportamento
- Múltiplas implementações
- Herança múltipla através de interfaces

3 Polimorfismo Avançado

- Polimorfismo paramétrico
- Generics
- Padrões de projeto

Leitura Recomendada

- **Microsoft Docs:** Inheritance in C#
- **Clean Code:** Robert C. Martin
- **Effective C#:** Bill Wagner
- **Pro C# 10:** Andrew Troelsen

Prática Recomendada:

- 1 Implemente os exemplos da pasta Exemplos
- 2 Resolva os exercícios propostos
- 3 Crie suas próprias hierarquias de classes
- 4 Pratique polimorfismo com diferentes tipos

Dúvidas e Discussão

Obrigado pela atenção!

Pontos-Chave da Aula:

- 1 **Herança** permite reutilização de código e organização hierárquica
- 2 **Classe derivada** herda membros da classe base
- 3 **Métodos virtuais** permitem sobrescrita e polimorfismo
- 4 **Construtores** devem chamar construtor da base
- 5 **Herança** modela relacionamento "é um"